

Lab 02- Docker

Applications Web

Dans cet atelier, vous allez déployer un serveur web et mettre en œuvre la technique de load-balancing pour améliorer le temps de réponse. Par la suite vous allez composer plusieurs conteneurs pour mettre en place une application dotée d'une architecture complexe.

Concepts : Dockerfile, Volume, Load-balancer, et Docker compose.

Important

Pour chaque exercice, créer un nouveau répertoire (exemple `q1`), travailler dans ce répertoire qui lui correspond. Mettre les commandes et les réponses dans les fichiers `reponses.txt`. Le code source, les fichiers Docker et les fichiers de configurations doivent tous figurer dans le répertoire qui correspond à l'exercice en question.

Compresser tous vos répertoires et fichiers dans une archive. Nommez votre archive `"INFO1_FamilyName_FirstName.txt"` (selon le groupe INFO1 ou INFO2).

1. Exécuter votre premier serveur web
 - 1.1. Créer une page web qui affiche la date d'aujourd'hui dans le format suivant (en Français):
"Bonjour, aujourd'hui c'est le Lundi, 25 Septembre 2026"
 - 1.2. Exécuter un conteneur Docker contenant le serveur web Apache qui utilise la page web que vous venez de créer comme page index.
 - 1.3. Créer une image Docker nommée "web-server" à l'aide d'un fichier Dockerfile.
 - 1.4. En utilisant le fichier Dockerfile exécuter votre serveur Web en mode daemon.
 - 1.5. Arrêter le serveur web
 - 1.6. Démarrer le serveur web de nouveau

2. Performances du serveur Web

- 2.1. Créer une image Docker nommée “benchmarker” intégrant l'outil de test de performance Apache “HTTP server benchmarking tool”.
- 2.2. Utiliser l'image “benchmarker” pour tester les performances de votre serveur Web.
- 2.3. Utilisez setTimeout() dans votre code Javascript pour retarder l'affichage du message de 3 secondes.
- 2.4. Testez les performances de nouveau. Qu'est ce qui a changé?

3. Volumes

- 3.1. En utilisant le fichier Dockerfile exécuter 2 instances de votre serveur Web en mode daemon.
- 3.2. En utilisant les commandes docker ajouter le texte “Instance 1” dans la page index de la première instance, et “Instance 2” dans la deuxième. Rafraichissez la page web plusieurs fois dans votre navigateur. Qu'est ce que vous observez?
- 3.3. Créez un volume et utilisez le pour partager la page web entre les deux instances du serveur web.

4. Load balancing

- 4.1. Utilisez une image NGINX pour implémenter une solution de load-balancing.
- 4.2. Testez les performances de nouveau. Qu'est ce qui a changé?
- 4.3. Créer un fichier docker-compose.yml permettant de répliquer votre architecture actuelle (4 conteneurs: 2 instances du serveur web, une instance NGINX et une instance “benchmarker”). Testez votre solution.

5. Back-end

- 5.1. Créez un service web en Python qui renvoie la date d'aujourd'hui dans le format suivant (en Français):
"Lundi, 25 Septembre 2026"
- 5.2. Exécuter un conteneur Docker contenant votre service web. Testez.
- 5.3. Modifiez votre page web afin de consommer le service web offert par le conteneur Docker.
- 5.4. Mettez à jour votre fichier docker-compose.yml afin d'inclure le service web.
- 5.5. Testez les performances de nouveau. Qu'est ce qui a changé?